

Coredeux Import Reference

[Previous: Text And Excel File Import](#) | [Documentation Home](#) | [Next: Coredeux Export](#)

This document explains the current `coredeux-import` module in enough detail for a developer or agent to use it, extend it, test it, or debug it without needing the original implementation chat.

Purpose

`coredeux-import` executes structured import requests against Coredeux-managed entities. The module is intentionally based on a JSON/DTO contract. It also ships the text and Excel parsers that compile import files into that JSON/DTO contract.

Supported today:

- JSON/DTO import request execution.
- POJO/entity imports backed by `CoredeuxService`.
- CREATE, UPSERT, MODIFY, DELETE, and FETCH.
- Multi-pass execution for row-key references.
- Simple macros and default values.
- Type conversion for common scalar types, enums, primitive collections, and entity references.
- Row-level error collection.
- Collection write modes: `replace`, `append`, and `clear`.
- Text file parsing through `CoredeuxTextImportParser`.
- Workbook parsing through `CoredeuxExcelImportParser`.

Not supported directly by `coredeux-import` today:

- CSV as a standalone format and multipart uploads.
- Nested object or array conversion by the default handler.
- Partial collection removal by value.

The intended architecture is:

```
coredeux-import
  text and workbook parsing
    -> ImportRequest JSON/DTO contract
    -> validation + execution through Coredeux core services
```

See [Text And Excel File Import](#) for the text and workbook parsers that convert import files into `ImportRequest`.

Module Location

Main code lives under:

```
modules/coredeux-import/src/main/java/com/coredeux/impex
```

Tests live under:

```
modules/coredeux-import/src/test/java/com/coredeux/impex
```

Demo integration lives in:

```
examples/coredeux-spring-boot-demo  
examples/coredeux-java-native-demo
```

The Spring Boot demo exposes:

- POST /api/import/validate
- POST /api/import

See:

- examples/coredeux-spring-boot-demo/src/main/java/com/coredeux/demo/web/CoredeuxImportController.java
- examples/coredeux-spring-boot-demo/postman/coredeux-spring-boot-demo-import.postman_collection.json

The native demo shows the same import engine without Spring:

- examples/coredeux-java-native-demo/src/main/java/com/coredeux/examples/nativejava/postgres/PostgresCustomerImportMain.java
- examples/coredeux-java-native-demo/src/main/resources/samples/postgres-customers.import

Dependency Model

coredeux-import depends directly on:

- coredeux-core
- Jackson databind
- Apache POI
- Lombok

Spring Boot integration lives in `coredeux-import-spring-boot-starter`.

It does not depend on a persistence implementation directly. Persistence comes from the application through `CoredeuxService`, which in turn uses the configured Coredeux data access service, such as `coredeux-core-jpa`.

Applications using the import module must include:

- coredeux-import
- coredeux-core

- a persistence adapter module such as `coredeux-core-jpa`
- entity definitions in the normal Coredeux registry

Example module dependency:

```
<dependency>
  <groupId>com.coredeux</groupId>
  <artifactId>coredeux-import</artifactId>
  <version>${project.version}</version>
</dependency>
```

High-Level Package Structure

```
com.coredeux.impex
  exception
    CoredeuxImportException
  handler
    CoredeuxImportValueHandler
    ImportValueContext
    ImportValueHandlerResolver
  handler.impl
    DefaultCoredeuxImportValueHandler
    JsonMapImportHandler
  model
    ImportColumn
    ImportLog
    ImportMacro
    ImportOperation
    ImportOptions
    ImportRequest
    ImportResponse
    ImportRow
    ImportSeverity
    ImportStatement
  service
    CoredeuxImportService
  service.impl
    DefaultCoredeuxImportService
    ImportEntityMetadata
    ImportEntityTargetService
```

Public API

The main service interface is `CoredeuxImportService`.

```
public interface CoredeuxImportService {
    ImportResponse validateData(ImportRequest request);
    ImportResponse importData(ImportRequest request);
}
```

Use `validateData` to validate the request shape and target mapping without executing rows.

Use `importData` to validate and execute rows. If validation fails, execution does not start and the response contains error logs.

JSON Contract Overview

The root request object is `ImportRequest`.

```
{
  "macros": {
    "&status": {
      "value": "ACTIVE"
    }
  },
  "options": {
    "passes": 2,
    "failFast": false,
    "validateOnly": false
  },
  "statements": [
    {
      "operation": "UPSERT",
      "entity": "com.example.Product",
      "columns": [
        {
          "name": "sku",
          "unique": true
        },
        {
          "name": "name"
        },
        {
          "name": "price",
          "defaultValue": "0"
        }
      ],
      "rows": [
        {
          "key": "&product1",
          "values": {
            "sku": "SKU-1",
            "name": "Demo Product",
            "price": "19.99"
          }
        }
      ]
    }
  ]
}
```

ImportRequest

Class: `ImportRequest`

Fields:

- `macros`: map of macro name to `ImportMacro`.
- `options`: request-level `ImportOptions`.
- `statements`: ordered list of `ImportStatement` blocks.

Statements execute in request order on every pass.

ImportOptions

Class: `ImportOptions`

Fields:

- `passes`: number of execution passes. Default is 2.
- `failFast`: when `true`, stop after the first final-pass row error.
- `validateOnly`: when `true`, `importData` performs validation but skips row execution and persistence.

Use multiple passes when rows refer to keys created by later statements or later rows.

ImportStatement

Class: `ImportStatement`

Fields:

- `operation`: one of `CREATE`, `UPSERT`, `MODIFY`, `DELETE`, `FETCH`.
- `entity`: the fully qualified Java class name.
- `columns`: ordered `ImportColumn` definitions.
- `lookup`: optional statement-level lookup definitions for `UPSERT`, `MODIFY`, `DELETE`, and `FETCH`.
- `query`: optional store-specific query definition for `UPSERT`, `MODIFY`, `DELETE`, and `FETCH`.
- `rows`: ordered `ImportRow` values.
- `metadata`: opaque extension map. See "Metadata Fields".

For non-`CREATE` operations, the statement must define exactly one existing-entity resolution strategy:

- at least one `unique: true` column
- a non-empty `lookup` list
- a `query`

Do not combine these strategies in the same statement. Strict validation is intentional because import execution must identify one existing entity through one clear path.

ImportLookup

Class: `ImportLookup`

Fields:

- `field`: target search field used in `SearchParams.field`.
- `comparator`: search comparator used in `SearchParams.comparator`. Defaults to

EQUALS.

- `column`: optional source import column used to derive the lookup value. When omitted, Coredeux resolves the source column from `field`.
- `nullSearch`: when the derived lookup value is null, use `ISNULL` instead of the configured comparator.
- `metadata`: opaque extension map. See "Metadata Fields".

Lookup resolution:

1. `field` always means the target field to search.
2. If `column` is present, Coredeux derives the lookup value from that import column.
3. If `column` is omitted, Coredeux finds a column whose `name` matches `field`.
4. The lookup value is derived through the normal column pipeline: row value, default value, macro replacement, custom handler, type conversion, and reference resolution.
5. The generated `SearchParams` uses `field`, the configured comparator, and the derived value.

Common case:

```
{
  "field": "sku",
  "comparator": "EQUALS"
}
```

This searches target field `sku` using the derived value from the import column that also targets `sku`.

Use `column` only when the lookup source and target field differ.

Example:

```
{
  "field": "emailHash",
  "comparator": "EQUALS",
  "column": "email"
}
```

This searches target field `emailHash` using the derived value from import column `email`. A realistic use case is an `email` column with a custom handler that normalizes and hashes the email address; the database stores only the derived `emailHash`, so the lookup must compare against `emailHash` even though the source import data is `email`.

ImportQuery

Class: `ImportQuery`

Fields:

- `text`: store-specific query text passed to `CoredeuxService.query`.

- **params**: map of query parameter name to `ImportQueryParam`.
- **metadata**: opaque extension map. See "Metadata Fields".

The import module does not parse, translate, or inline query values. It derives the parameter values and calls:

```
coredeuxService.query(query.text, derivedParams, targetClass, -1, -1)
```

The selected `CoredeuxDataAccessService` decides how to interpret the query text and parameter map for SQL, NoSQL, search engines, or any other backend.

ImportQueryParam

Class: `ImportQueryParam`

Fields:

- **column**: optional source import column used to derive the query parameter value. When omitted, the query parameter name is used as the source column name.
- **metadata**: opaque extension map. See "Metadata Fields".

Query parameter resolution:

1. The query parameter map key is the parameter name passed to `CoredeuxService.query`.
2. If **column** is present, Coredeux derives the parameter value from that import column.
3. If **column** is omitted, Coredeux derives the parameter value from the import column whose name matches the parameter name.
4. The value is derived through the normal column pipeline: row value, default value, macro replacement, custom handler, type conversion, and reference resolution.

Documentation placeholder style:

```
{{paramName}}
```

Coredeux docs use `{{paramName}}` to make examples readable and visibly separate parameter names from store syntax. This is not runtime syntax owned by `coredeux-import`. The import module does not replace `{{paramName}}`, does not translate it to `:paramName`, and does not inline parameter values.

At runtime, write `query.text` in the syntax expected by the configured `CoredeuxDataAccessService`. For example, a JPA-backed implementation may expect `:paramName`, while a MongoDB or search adapter may expect a JSON/DSL shape. `coredeux-import` only derives the parameter map and passes both values through.

ImportColumn

Class: `ImportColumn`

Fields:

- **name**: logical column name. Row values are keyed by this name.
- **unique**: marks the column as part of the lookup key for UPSERT, MODIFY, DELETE, and FETCH.
- **nullSearch**: only applies to **unique: true** lookup columns. When the converted lookup value is null, use the ISNULL comparator instead of EQUALS.
- **defaultValue**: used when the row value is null or blank.
- **mode**: collection update mode for POJO collection fields. Supported values are **replace**, **append**, and **clear**; default is **replace**.
- **reference**: reference lookup instruction. See "References".
- **handler**: Spring bean name of a custom value handler.
- **metadata**: opaque extension map. See "Metadata Fields".

Column name is both the row value key and the target entity field name.

ImportRow

Class: `ImportRow`

Fields:

- **key**: optional row key such as `&product1`. Later rows can reference it with **reference: "*"** .
- **values**: map of column name to raw value.
- **metadata**: opaque extension map. See "Metadata Fields".

Rows are tracked by object identity during multi-pass execution. This matters because two rows with identical values are still distinct rows and both should be processed.

ImportMacro

Class: `ImportMacro`

Fields:

- **value**: replacement value.
- **metadata**: opaque extension map. See "Metadata Fields".

Macro replacement is simple: if the effective string value starts with `&` and matches a key in `ImportRequest.macros`, it is replaced with the macro value.

Example:

```
{
  "macros": {
    "&activeStatus": { "value": "ACTIVE" }
  },
  "statements": [
    {
```



```

        "operation": "CREATE",
        "entity": "com.example.Customer",
        "columns": [
            { "name": "status" }
        ],
        "rows": [
            { "values": { "status": "&activeStatus" } }
        ]
    }
}

```

Metadata Fields

Several import DTOs expose a `metadata` map. The import engine treats this map as opaque extension data. It stores the values on the request model and makes them available to code that receives the model, but the core import engine does not validate, interpret, or execute behavior from metadata keys.

Use `metadata` when a custom handler, parser, or application-specific extension needs extra configuration without adding a first-class field to the public import schema.

Column metadata is the most common handler configuration use case:

```

{
    "name": "sku",
    "handler": "skuNormalizeImportHandler",
    "metadata": {
        "trim": true,
        "case": "upper",
        "prefix": "SHOP-"
    }
}

```

The import engine does not know what `trim`, `case`, or `prefix` mean. A custom handler can read them from the current column and decide how to transform the value:

```

@Component("skuNormalizeImportHandler")
public class SkuNormalizeImportHandler implements CoredeuxImportValueHandler {

    @Override
    public Object handle(ImportValueContext context) {
        Map<String, Object> metadata = context.getColumn().getMetadata() == null
            ? Map.of()
            : context.getColumn().getMetadata();
        String value = context.getEffectiveValue();

        if (Boolean.TRUE.equals(metadata.get("trim")) && value != null) {
            value = value.trim();
        }
        if ("upper".equals(metadata.get("case")) && value != null) {
            value = value.toUpperCase(Locale.ROOT);
        }
        if (metadata.get("prefix") instanceof String prefix && value != null) {
            value = prefix + value;
        }
        return value;
    }
}

```

Good metadata examples:

- handler options such as date format, case policy, fallback mode, or parser hints.
- trace information from a file parser, such as source sheet name or original header.
- application-specific flags consumed by a known handler.

Avoid using metadata for behavior that every importer should understand. If a setting changes core import execution, validation, entity resolution, reference resolution, or persistence semantics, it should become an explicit DTO field instead of a metadata key.

ImportResponse

Class: `ImportResponse`

Fields:

- `logs`: list of `ImportLog`.

`ImportResponse` intentionally does not expose internal row-key references. References are kept in a private execution map inside `DefaultCoredeuxImportService.importData`.

`hasErrors()` returns `true` if any log has severity `ERROR` or `CRITICAL`.

ImportLog

Class: `ImportLog`

Fields:

- `severity`: `SUCCESS`, `INFO`, `WARN`, `ERROR`, or `CRITICAL`.
- `message`: human-readable diagnostic.
- `statementIndex`: one-based statement index when available.
- `rowIndex`: one-based row index when available.
- `entity`: entity name/class involved.
- `column`: column name when available.
- `exceptionType`: exception class name.

Statement-level validation errors have a `statementIndex` but usually no `rowIndex`, because row processing has not started.

Example validation error:

```
{
  "logs": [
    {
      "severity": "ERROR",
      "message": "At least one unique column is required for operation: MODIFY",
      "statementIndex": 1,
      "rowIndex": null,
      "entity": "com.coredeux.demo.domain.Product",
      "column": null,
    }
  ]
}
```

```
    "exceptionType": "com.coredeux.impex.exception.CoredeuxImportException"  
  }  
]  
}
```

Operations

CREATE

Creates a new target object for every row.

Flow:

1. Resolve target metadata.
2. Create an instance using the no-argument constructor.
3. Convert and write all columns.
4. Save using `CoredeuxService.save`.
5. Store row key internally when `row.key` is present.

CREATE does not require unique columns.

UPSERT

Updates an existing object when unique-column lookup finds one; otherwise creates a new object.

Flow:

1. Convert unique column values.
2. Search with `CoredeuxService.loadAll`.
3. If no result, execute create flow.
4. If one result, populate the existing object and call `CoredeuxService.update`.
5. If more than one result, log an error.

At least one column must have `unique: true`.

MODIFY

Requires an existing object.

Flow:

1. Search by unique columns.
2. If no result, log an error.
3. If one result, populate the existing object and call `CoredeuxService.update`.
4. If more than one result, log an error.

At least one column must have `unique: true`.

DELETE

Deletes an existing object when found by unique columns.

Flow:

1. Search by unique columns.
2. If found, call `CoredeuxService.remove(existing)`.
3. If not found, no-op.

At least one column must have `unique: true`.

FETCH

Fetches an existing object and stores its row key internally for later `reference: "*" resolution`.

Flow:

1. Search by unique columns.
2. If not found, log an error.
3. If found, store `row.key -> identifier` internally.

At least one column must have `unique: true`.

Execution Flow

Implementation class: `DefaultCoredeuxImportService`

validateData

`validateData(request)` creates an empty response and calls internal `validate`.

Validation checks:

- request is not null.
- request contains at least one statement.
- each statement is not null.
- statement operation is not null.
- statement has at least one column.
- row list is initialized if null.
- entity target service can resolve the entity class and field mappings.
- column definitions are not null.
- column names are not blank.
- column names are unique within a statement.
- entity columns map to real Java fields.

- row keys are either omitted or non-blank.
- collection mode values are valid.
- append and clear collection modes are used only on collection fields.
- non-CREATE operations have exactly one resolution strategy.

importData

`importData(request):`

1. Creates a response.
2. Validates the request.
3. Returns immediately when validation fails.
4. Returns immediately when `options.validateOnly` is true.
5. Creates an internal `Map<String, String>` for row-key references.
6. Executes statements and rows for `options.passes`.
7. Skips rows already imported successfully in prior passes.
8. Collects row errors only on the final pass.
9. Honors `options.failFast` on final-pass row errors.

Errors during earlier passes are intentionally suppressed to allow forward references to resolve on later passes.

References

References let a column value resolve to another entity instance instead of a plain scalar.

A reference is enabled by setting `ImportColumn.reference`.

Single-Field Reference

Example:

```
{
  "name": "product",
  "reference": "sku"
}
```

If the target field type is `Product`, and the row value is `SKU-1`, the default handler searches:

```
Product where sku EQUALS "SKU-1"
```

The result must contain exactly one entity.

Compound Reference

Example:

```
{
```

```
"name": "product",
"reference": "sku:category"
}
```

With row value:

```
SKU-1:SOFTWARE
```

The default handler searches:

```
Product where sku EQUALS "SKU-1" and category EQUALS SOFTWARE
```

Values are split by `:`. If fewer values than reference fields are supplied, the row fails. Use `\:` when a reference value segment itself contains a colon, and `\\` when it contains a literal backslash.

Example with an escaped colon:

```
SKU\:A:SOFTWARE
```

For reference: `"sku:category"`, this produces:

```
sku = SKU:A
category = SOFTWARE
```

Row-Key Reference

Example:

```
{
  "name": "customer",
  "reference": "*"
}
```

With row value:

```
&customer1
```

The service resolves `&customer1` from the internal reference map populated by earlier successful rows in the same import execution. It then loads the target entity by identifier using `CoredeuxService.load`.

Row-key references are internal. They are not returned in `ImportResponse`.

Collection References

When the target field is a collection, the default handler splits the effective value by commas and resolves each element. Use backslash escaping when a value itself contains a comma or backslash:

- `\,` becomes a literal comma.
- `\\` becomes a literal backslash.

Example:

```
{
  "name": "roles",
  "reference": "code"
}
```

With row value:

```
BUYER, REVIEWER
```

If the target field is `Set<Role>`, this resolves both roles by `code` and returns a `LinkedHashSet`.

Example with an escaped comma:

```
sku\,with-comma,sku-normal
```

This produces two collection entries:

```
sku,with-comma  
sku-normal
```

Collection Modes

Collection columns use `mode` to decide how incoming collection values are written to an existing POJO target.

`replace` is the default:

```
{  
  "name": "roles",  
  "reference": "code",  
  "mode": "replace"  
}
```

When importing into an existing entity, the current collection is replaced by the imported collection.

`append` preserves the existing collection and adds incoming values:

```
{  
  "name": "roles",  
  "reference": "code",  
  "mode": "append"  
}
```

If the existing collection is null, the entity target service creates a compatible collection for common field types such as `Set` and `List`.

`clear` empties the current collection without needing an incoming value:

```
{  
  "name": "roles",  
  "mode": "clear"  
}
```

If the existing collection is null, `clear` initializes an empty compatible collection instead of leaving the field null.

There is no separate `merge` mode because `append` already means "add incoming values to the existing collection". Partial `remove` semantics are intentionally not supported yet because matching collection elements reliably depends on the element type and domain identity rules.

Unsupported modes fail with `CoredeuxImportException`.

Default Value Conversion

Implementation class: `DefaultCoredeuxImportValueHandler`

The default handler is registered as:

`coredeuxDefaultImportValueHandler`

Supported conversions from string:

- `String`
- `boolean / Boolean`
- `int / Integer`
- `long / Long`
- `short / Short`
- `byte / Byte`
- `double / Double`
- `float / Float`
- `BigInteger`
- `BigDecimal`
- `UUID`
- `LocalDate`
- `LocalDateTime`
- `Instant`
- `OffsetDateTime`
- `enums`
- collections of supported element types
- references and reference collections

Unsupported by the default handler:

- `char / Character`
- `LocalTime`
- `OffsetTime`
- `ZonedDateTime`
- `Year, YearMonth, MonthDay`
- `Duration, Period`
- legacy date/time types such as `Date`, `Calendar`, `Timestamp`, `java.sql.Date`, and

`java.sql.Time`

- URI, URL
- Locale, Currency, ZoneId, ZoneOffset
- arrays
- maps
- nested POJOs or arbitrary complex object construction

For these types, implement a custom `CoredeuxImportValueHandler` and reference it from `ImportColumn.handler`. This keeps the default handler deterministic and prevents import from becoming a general-purpose object mapper.

Blank or null values become:

- `null` for object types.
- primitive defaults for primitive fields, such as `false` or `0`.

Unsupported conversions throw `CoredeuxImportException`.

The default handler does not convert `Map` fields. Use `jsonMapImportHandler` explicitly for JSON object or map columns.

Custom Value Handlers

Custom handlers implement:

```
public interface CoredeuxImportValueHandler {  
    Object handle(ImportValueContext context);  
}
```

Register handlers as Spring beans. Use the bean name in `ImportColumn.handler`.

Example:

```
@Component("uppercaseHandler")  
public class UppercaseImportHandler implements CoredeuxImportValueHandler {  
    @Override  
    public Object handle(ImportValueContext context) {  
        return context.getEffectiveValue() == null  
            ? null  
            : context.getEffectiveValue().toUpperCase();  
    }  
}
```

JSON:

```
{  
  "name": "sku",  
  "handler": "uppercaseHandler"  
}
```

Unsupported Type Handler Example

The demo application includes a custom handler for `java.net.URI`, which is not converted by the default handler. The `Product` entity exposes:

```
private URI documentationUrl;
```

The demo persists that field through a normal JPA converter:

```
@Converter(autoApply = true)
public class UriAttributeConverter implements AttributeConverter<URI, String> {

    @Override
    public String convertToDatabaseColumn(URI attribute) {
        return attribute == null ? null : attribute.toString();
    }

    @Override
    public URI convertToEntityAttribute(String dbData) {
        return dbData == null || dbData.isBlank() ? null : URI.create(dbData);
    }
}
```

Persistence and import conversion are separate concerns. The JPA converter tells the database layer how to store a `URI`; the import handler tells Coredeux how to turn an incoming row value into a `URI`.

The import handler is registered as a Spring bean:

```
@Component("demoUriImportHandler")
public class DemoUriImportHandler implements CoredeuxImportValueHandler {

    @Override
    public Object handle(ImportValueContext context) {
        String value = context.getEffectiveValue();
        String columnName = context.getColumn() == null ? null :
context.getColumn().getName();
        if (value == null || value.isBlank()) {
            return null;
        }
        URI uri;
        try {
            uri = URI.create(value.trim());
        } catch (IllegalArgumentException exception) {
            throw new CoredeuxImportException("Invalid URI value for column: " +
columnName, columnName, exception);
        }
        if (!uri.isAbsolute()) {
            throw new CoredeuxImportException("URI value must be absolute for
column: " + columnName, columnName);
        }
        return uri;
    }
}
```

The column opts into the handler by bean name:

```
{
  "name": "documentationUrl",
  "handler": "demoUriImportHandler"
}
```

Example row:

```
{
  "values": {
    "sku": "IMPORT-SKU-1001",
    "documentationUrl": "https://docs.coredeux.dev/products/import-starter"
  }
}
```

Handler errors should use `CoredeuxImportException` with the column name. That lets `ImportLog.column` point to the failing column instead of reporting only a row-level failure.

jsonMapImportHandler

The project ships a ready-to-use handler bean named `jsonMapImportHandler` that parses and validates Map/JSON column values. Use it when your import data contains JSON objects (either as already-parsed Map values produced by a file parser, or as JSON strings). Map handling is opt-in; set `ImportColumn.handler` to `jsonMapImportHandler` for every map column.

Behavior:

- If the raw row value is already a `Map<?, ?>`, the handler validates that all keys are strings and converts values according to the target field's generic map value type.
- If the raw row value is a `String`, the handler attempts to parse it as a JSON object using Jackson (`ObjectMapper`) and then applies the same value conversion.
- Blank strings return `null`.
- JSON strings must contain an object. Arrays and primitive JSON values are rejected.
- Map keys must be strings.
- Map values must be simple scalar values. Nested objects and arrays are rejected by this handler.
- When keys are not strings, JSON parsing fails, the JSON root is not an object, or a value cannot be converted, the handler throws a `CoredeuxImportException` with the column name so validation and execution logs report the column-level error.

Supported map value conversion:

- `Map<String, Object>` preserves scalar `String`, `Number`, `Boolean`, and `Character` values as supplied by the parser/Jackson.
- `Map<String, String>` converts scalar values to strings. For example, JSON 42 becomes "42".
- `Map<String, T>` converts scalar values to T when T is one of: `Boolean`, `Integer`, `Long`, `Short`, `Byte`, `Double`, `Float`, `BigInteger`, `BigDecimal`, `UUID`, `LocalDate`, `LocalDateTime`, `Instant`, `OffsetDateTime`, or an enum.
- Raw or unparameterized map targets use `Object` behavior.

Unsupported by `jsonMapImportHandler`:

- non-string map keys
- nested map/object values
- array or collection values
- `char` / `Character` as a declared generic value type
- the same date/time and miscellaneous Java types unsupported by the default handler, such as `LocalTime`, `ZonedDateTime`, `Duration`, `Date`, `URI`, `Locale`, `Currency`, and `ZoneId`
- domain objects or arbitrary complex value types

For unsupported map value types, implement a custom handler. The custom handler can read `context.getRawValue()`, `context.getEffectiveValue()`, `context.getMapValueType()`, and column metadata to parse and validate the value using application-specific rules.

Usage:

Set the import column's `handler` field to `jsonMapImportHandler`:

```
{
  "name": "metadata",
  "handler": "jsonMapImportHandler"
}
```

Parser expectations and compatibility:

- The handler accepts both already-parsed Map objects (recommended for parser modules that can convert JSON cells into objects) and JSON strings. If your parser produces a JSON string for object columns, the handler will parse it.
- Mixed input is deterministic: a parsed map row and a JSON string row for the same column both end as a converted `Map<String, ?>` using the target field's declared generic value type.
- If you need nested object support or domain-specific values, implement a custom handler.

Examples:

Row values with parsed map (parser produced a Map):

```
{
  "columns": [
    { "name": "metadata", "handler": "jsonMapImportHandler" }
  ],
  "values": {
    "sku": "sku-1",
    "metadata": { "family": "demo", "priority": 7 }
  }
}
```

Row values with JSON string (handler will parse):

```
{
  "columns": [
    { "name": "metadata", "handler": "jsonMapImportHandler" }
  ]
}
```

```

    ],
    "values": {
      "sku": "sku-1",
      "metadata": "{\"family\":\"demo\", \"priority\":7}"
    }
  }
}

```

Example with a declared generic value type:

```

private Map<String, Integer> ratings;

{
  "columns": [
    { "name": "ratings", "handler": "jsonMapImportHandler" }
  ],
  "rows": [
    {
      "values": {
        "ratings": {
          "quality": "10",
          "support": 8
        }
      }
    }
  ]
}

```

The handler sees `mapValueType == Integer.class` and stores:

```

{
  "quality": 10,
  "support": 8
}

```

Use the `jsonMapImportHandler` when you want convenient JSON object handling in imports; implement a custom handler when you need domain-specific conversion or validation.

legacyDateImportHandler (java.util.Date)

The default import handler does not convert into `java.util.Date`. When you still have legacy models or external contracts that require `Date`, implement a custom handler and pass a pattern through column metadata.

The demo includes such a handler as a reference implementation:

- bean name: `legacyDateImportHandler`
- target type: `java.util.Date`
- required metadata: `metadata.dateFormat`
- optional metadata: `metadata.timezone`

Example column:

```

{
  "name": "legacySignupDate",
  "handler": "legacyDateImportHandler",
  "metadata": {
    "dateFormat": "dd/MM/yyyy",

```

```

    "timezone": "UTC"
  }
}

```

Example implementation (from `examples/coredeux-spring-boot-demo`):

```

package com.coredeux.demo.imports;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Locale;
import java.util.Map;
import java.util.TimeZone;

import org.springframework.stereotype.Component;

import com.coredeux.impex.exception.CoredeuxImportException;
import com.coredeux.impex.handler.CoredeuxImportValueHandler;
import com.coredeux.impex.handler.ImportValueContext;

@Component("legacyDateImportHandler")
public class LegacyDateImportHandler implements CoredeuxImportValueHandler {

    @Override
    public Object handle(ImportValueContext context) {
        String value = context.getEffectiveValue();
        String columnName = context.getColumn() == null ? null :
context.getColumn().getName();
        if (value == null || value.isBlank()) {
            return null;
        }
        if (context.getExpectedType() != null && !
Date.class.isAssignableFrom(context.getExpectedType())) {
            throw new CoredeuxImportException("legacyDateImportHandler supports
java.util.Date targets only", columnName);
        }

        Map<String, Object> metadata = context.getColumn() == null ? null :
context.getColumn().getMetadata();
        String pattern = metadata == null ? null :
stringValue(metadata.get("dateFormat"));
        if (pattern == null || pattern.isBlank()) {
            throw new CoredeuxImportException("Missing metadata.dateFormat for
legacy date column: " + columnName, columnName);
        }

        SimpleDateFormat format = new SimpleDateFormat(pattern, Locale.ROOT);
        format.setLenient(false);
        String timezone = metadata == null ? null :
stringValue(metadata.get("timezone"));
        if (timezone != null && !timezone.isBlank()) {
            format.setTimeZone(TimeZone.getTimeZone(timezone));
        }
        try {
            return format.parse(value.trim());
        } catch (ParseException exception) {
            throw new CoredeuxImportException("Invalid date value for column: "
+ columnName, columnName, exception);
        }
    }

    private String stringValue(Object value) {

```

```

        return value == null ? null : String.valueOf(value);
    }
}

```

ImportValueContext

Class: `ImportValueContext`

Fields:

- `rawValue`: original row value.
- `effectiveValue`: string after default value and macro replacement.
- `expectedType`: target field type.
- `collectionElementType`: collection generic element type when available.
- `mapValueType`: map generic value type when available.
- `targetEntityType`: target Java type.
- `column`: current `ImportColumn`.
- `row`: current `ImportRow`.
- `statement`: current `ImportStatement`.
- `macros`: request macros.
- `references`: internal row-key reference map.

Handlers are Spring beans, so inject dependencies through constructors. Do not expect service dependencies to be placed in `ImportValueContext`.

The service validates handler output before writing it to the target. If a handler returns a value incompatible with the target field type, the row fails.

Entity Target Service

Entity-specific behavior is isolated from the core import orchestration in `ImportEntityTargetService`.

Supporting target metadata is held in `ImportEntityMetadata`.

```

ImportEntityMetadata resolveTarget(ImportStatement statement);

Object createInstance(ImportEntityMetadata metadata);

void writeValue(Object target, ImportColumn column, Object value,
ImportEntityMetadata metadata);

Object identifier(Object target, ImportEntityMetadata metadata);

```

`DefaultCoredeuxImportService` uses this service directly. The active contract supports entity imports only.

Responsibilities:

- Resolves `statement.entity` as a Java class.
- Resolves entity identifier from `EntityDefinitionRegistry`, defaulting to `id` if no definition exists.
- Validates row keys are not blank when supplied.
- Validates column names are present and unique.
- Validates each column name maps to a Java field.
- Validates collection mode values before row execution.
- Requires exactly one existing-entity resolution strategy for all non-CREATE operations: unique columns, statement lookup, or query.
- Validates lookup target fields and lookup source columns.
- Validates query text and query parameter source columns.
- Creates instances using the no-arg constructor.
- Writes values through `CoredeuxReflectionHelperService`.
- Reads identifiers for row-key storage.

Entity target fields can be inherited; field lookup uses `Coredeux` reflection helpers where applicable.

Existing-Entity Resolution

Non-CREATE operations need a way to find the existing entity they should modify, delete, fetch, or upsert. `Coredeux` supports three strategies.

Strategy 1: Unique Columns

Use this for simple equality or null checks.

```
{
  "columns": [
    {
      "name": "sku",
      "unique": true
    }
  ]
}
```

The service uses `CoredeuxService.loadAll` with generated `SearchParams`.

Unique-column comparators:

- `EQUALS`
- `ISNULL`

Unique lookup behavior:

- For each `unique: true` column, the column value is converted first.
- If converted value is null and `nullSearch` is true, comparator is `ISNULL`.

- Otherwise comparator is `EQUALS`.
- No result:
 - allowed for `UPSERT` and `DELETE`.
 - error for `MODIFY` and `FETCH`.
- More than one result is always an error.

Strategy 2: Statement Lookup

Use this when equality against `unique: true` columns is not enough, but the lookup can still be represented as structured `SearchParams`.

The service uses `CoredeuxService.loadAll` with generated `SearchParams`.

Statement lookup behavior:

- For each lookup entry, `Coredeux` resolves a source `ImportColumn`.
- If `lookup.column` is set, that column is used.
- If `lookup.column` is omitted, `Coredeux` finds a column whose name matches `lookup.field`.
- The lookup value is derived through the source column's normal handler and conversion pipeline.
- If derived value is null and `lookup.nullSearch` is true, comparator is `ISNULL`.
- Otherwise comparator is `lookup.comparator`, defaulting to `EQUALS`.
- The selected `CoredeuxDataAccessService` must understand the comparator. Use `CoredeuxService.supportedComparators(entityClass)` or `CoredeuxDataAccessService.supportedComparators(entityClass)` to discover the comparator names available for the target application.

Examples of comparators a data-access service may support:

- `EQUALS`
- `ISNULL`
- `STARTSWITH`
- `CONTAINS`
- store-specific custom comparators

Statement Lookup Examples

Use statement lookup when equality against `unique: true` columns is not enough.

Example with a comparator:

```
{
  "operation": "MODIFY",
  "entity": "com.example.Product",
```

```

"lookup": [
  {
    "field": "sku",
    "comparator": "STARTSWITH"
  }
],
"columns": [
  {
    "name": "sku"
  },
  {
    "name": "status"
  }
],
"rows": [
  {
    "values": {
      "sku": "ABC-",
      "status": "DISCONTINUED"
    }
  }
]
}

```

This resolves the Sku column value through the normal column pipeline and builds:

```

field: sku
comparator: STARTSWITH
value: ABC-

```

Example with a different source column:

```

{
  "operation": "MODIFY",
  "entity": "com.example.Customer",
  "lookup": [
    {
      "field": "emailHash",
      "comparator": "EQUALS",
      "column": "email"
    }
  ],
  "columns": [
    {
      "name": "email",
      "handler": "emailHashImportHandler"
    },
    {
      "name": "status"
    }
  ],
  "rows": [
    {
      "values": {
        "email": "john@example.com",
        "status": "ACTIVE"
      }
    }
  ]
}

```

This searches emailHash using the value derived from import column email. That is useful

when the import file contains a human-readable value, but the stored lookup field contains a normalized or hashed version of it.

Strategy 3: Statement Query

Use this when the existing entity can only be found through a store-specific query.

Example:

```
{
  "operation": "MODIFY",
  "entity": "com.example.Product",
  "query": {
    "text": "<backend query that uses skuPrefix and status parameters>",
    "params": {
      "skuPrefix": {
        "column": "sku"
      },
      "status": {}
    }
  },
  "columns": [
    {
      "name": "sku",
      "handler": "skuPrefixHandler"
    },
    {
      "name": "status"
    },
    {
      "name": "price"
    }
  ],
  "rows": [
    {
      "values": {
        "sku": "ABC-123",
        "status": "ACTIVE",
        "price": "99"
      }
    }
  ]
}
```

For each row, Coredeux derives:

```
skuPrefix = derived value from column sku
status = derived value from column status
```

Then it calls `CoredeuxService.query` with the query text and derived params.

The query text remains store-specific. A SQL/JPA data-access service may require named parameters such as `:skuPrefix` and `:status`. A search backend may interpret the text as its own DSL. Coredeux import does not parse or rewrite the query string.

Example with a different source column:

```
{
  "operation": "MODIFY",
  "entity": "com.example.Customer",
  "query": {
```

```

    "text": "<backend query that uses the emailHash parameter>",
    "params": {
      "emailHash": {
        "column": "email"
      }
    },
    "columns": [
      {
        "name": "email",
        "handler": "emailHashImportHandler"
      },
      {
        "name": "status"
      }
    ],
    "rows": [
      {
        "values": {
          "email": "john@example.com",
          "status": "ACTIVE"
        }
      }
    ]
  }
}

```

This derives the query parameter `emailHash` from import column `email`. The email handler can normalize and hash `john@example.com`; the query searches the stored hash field without exposing the hash in the import file.

Cardinality Rules

All three strategies follow the same result rules:

- No result:
 - allowed for UPSERT and DELETE.
 - error for MODIFY and FETCH.
- One result:
 - use the result as the existing entity.
- More than one result:
 - always an error.

Non-CREATE operations must identify at most one existing entity. If a search returns multiple rows, Coredeux cannot safely know which item to modify, delete, fetch, or upsert.

Strict Strategy Validation

Use only one existing-entity resolution strategy in a statement.

Valid:

```

{
  "operation": "MODIFY",
  "query": {
    "text": "<backend query that uses the sku parameter>",
    "params": {

```

```

        "sku": {}
    },
    "columns": [
        { "name": "sku" },
        { "name": "status" }
    ]
}

```

Invalid:

```

{
  "operation": "MODIFY",
  "query": {
    "text": "<backend query that uses the sku parameter>",
    "params": {
      "sku": {}
    }
  },
  "columns": [
    { "name": "sku", "unique": true },
    { "name": "status" }
  ]
}

```

The invalid example mixes `query` and `unique`. Coredeux rejects it because hidden precedence would make imports hard to reason about.

Null Unique Lookups

Use `nullSearch: true` when a nullable field is intentionally part of the lookup key and a missing row value should search for existing records where that field is null.

Example:

```

{
  "operation": "MODIFY",
  "entity": "com.coredeux.demo.domain.Product",
  "columns": [
    {
      "name": "sku",
      "unique": true,
      "nullSearch": true
    },
    {
      "name": "status"
    }
  ],
  "rows": [
    {
      "values": {
        "status": "DISCONTINUED"
      }
    }
  ]
}

```

In this example, `sku` is omitted in the row. Because it is a unique column and `nullSearch` is true, the lookup sent to `CoredeuxService.loadAll` uses:

```
field: sku
```

```
comparator: ISNULL  
value: null
```

Without `nullSearch: true`, the same `null` value uses `EQUALS`. That distinction matters because data-access implementations may treat `EQUALS null` differently from `ISNULL`.

Demo API

The Spring Boot demo wires `coredeux-import` into the app.

Controller:

```
modules/coredeux-spring-boot-demo/src/main/java/com/coredeux/demo/web/  
CoredeuxImportController.java
```

Endpoints:

```
POST /api/import/validate  
POST /api/import
```

Both consume `ImportRequest` JSON and return `ImportResponse`.

Response status:

- 200 OK when `response.hasErrors()` is false.
- 400 BAD_REQUEST when `response.hasErrors()` is true.

The demo Postman import collection is:

```
modules/coredeux-spring-boot-demo/postman/coredeux-spring-boot-demo-  
import.postman_collection.json
```

The native demo exposes the same parser-to-request flow without Spring:

```
examples/coredeux-java-native-demo/src/main/java/com/coredeux/examples/  
nativejava/postgres/PostgresCustomerImportMain.java  
examples/coredeux-java-native-demo/src/main/resources/samples/postgres-  
customers.import
```

It includes:

- validation of a complete import graph.
- execution of a complete import graph.
- failure probes for missing fields, missing existing-entity resolution strategy, bad enums, missing row-key references, multiple row errors, unsupported nested map conversion, and mixed resolution strategies.

Full Example

This example imports roles, a product, a customer, an address, an order, and an order item. It demonstrates:

- UPSERT.
- unique lookup columns.

- row keys.
- reference: "*".
- reference: "code".
- reference: "sku".
- primitive, enum, date/time, decimal, and collection conversions.

```
{
  "options": {
    "passes": 3,
    "failFast": false
  },
  "statements": [
    {
      "operation": "UPSERT",
      "entity": "com.coredeux.demo.domain.Role",
      "columns": [
        { "name": "code", "unique": true },
        { "name": "description" }
      ],
      "rows": [
        {
          "key": "&roleBuyer",
          "values": {
            "code": "BUYER",
            "description": "Buyer role imported from JSON"
          }
        }
      ]
    },
    {
      "operation": "UPSERT",
      "entity": "com.coredeux.demo.domain.Product",
      "columns": [
        { "name": "sku", "unique": true },
        { "name": "name" },
        { "name": "price" },
        { "name": "active" },
        { "name": "category" },
        { "name": "tags" }
      ],
      "rows": [
        {
          "key": "&product1",
          "values": {
            "sku": "IMPORT-SKU-1001",
            "name": "Imported Starter Product",
            "price": "149.99",
            "active": "true",
            "category": "SOFTWARE",
            "tags": "import,demo,starter"
          }
        }
      ]
    },
    {
      "operation": "UPSERT",
      "entity": "com.coredeux.demo.domain.Customer",
      "columns": [
        { "name": "email", "unique": true },

```

```

    { "name": "name" },
    { "name": "active" },
    { "name": "status" },
    { "name": "registeredAt" },
    { "name": "birthDate" },
    { "name": "loyaltyPoints" },
    { "name": "creditLimit" },
    { "name": "phoneNumbers" },
    { "name": "roles", "reference": "code" }
  ],
  "rows": [
    {
      "key": "&customerAlice",
      "values": {
        "email": "import.alice@example.com",
        "name": "Import Alice",
        "active": "true",
        "status": "ACTIVE",
        "registeredAt": "2026-05-02T02:00:00Z",
        "birthDate": "1991-05-20",
        "loyaltyPoints": "150",
        "creditLimit": "1500.00",
        "phoneNumbers": "+61-400-111-222,+61-400-333-444",
        "roles": "BUYER"
      }
    }
  ]
},
{
  "operation": "UPSERT",
  "entity": "com.coredeux.demo.domain.Address",
  "columns": [
    { "name": "customer", "reference": "*" },
    { "name": "line1", "unique": true },
    { "name": "type" },
    { "name": "city" },
    { "name": "country" },
    { "name": "primaryAddress" }
  ],
  "rows": [
    {
      "key": "&addressAliceHome",
      "values": {
        "customer": "&customerAlice",
        "line1": "12 Import Street",
        "type": "HOME",
        "city": "Sydney",
        "country": "Australia",
        "primaryAddress": "true"
      }
    }
  ]
},
{
  "operation": "UPSERT",
  "entity": "com.coredeux.demo.domain.CustomerOrder",
  "columns": [
    { "name": "customer", "reference": "*" },
    { "name": "shippingAddress", "reference": "*" },
    { "name": "status" },
    { "name": "createdAt", "unique": true },
    { "name": "totalAmount" },
    { "name": "notes" }
  ],

```



```

    "rows": [
      {
        "key": "&orderAlice",
        "values": {
          "customer": "&customerAlice",
          "shippingAddress": "&addressAliceHome",
          "status": "DRAFT",
          "createdAt": "2026-05-02T03:00:00Z",
          "totalAmount": "399.49",
          "notes": "Imported order,verify references"
        }
      }
    ],
  },
  {
    "operation": "UPSERT",
    "entity": "com.coredeux.demo.domain.OrderItem",
    "columns": [
      { "name": "order", "reference": "*", "unique": true },
      { "name": "product", "reference": "sku", "unique": true },
      { "name": "quantity" },
      { "name": "unitPrice" }
    ],
    "rows": [
      {
        "key": "&orderItemAliceStarter",
        "values": {
          "order": "&orderAlice",
          "product": "IMPORT-SKU-1001",
          "quantity": "2",
          "unitPrice": "149.99"
        }
      }
    ]
  }
]
}

```

Testing

Important tests:

```

modules/coredeux-import/src/test/java/com/coredeux/impex/service/impl/
DefaultCoredeuxImportServiceTest.java
modules/coredeux-spring-boot-demo/src/test/java/com/coredeux/demo/web/
CoredeuxImportControllerTest.java
modules/coredeux-spring-boot-demo/src/test/java/com/coredeux/demo/
CoredeuxDemoApplicationTest.java

```

The import service tests cover:

- missing POJO field validation.
- default type conversions.
- upsert by unique column.
- statement lookup without unique columns.
- statement query without unique columns.
- query and lookup values derived through custom handlers.

- rejection of multiple existing-entity resolution strategies in one statement.
- custom handlers.
- invalid handler output type.
- single references.
- collection references.
- escaped commas in primitive collections and reference collections.
- `validateOnly`.
- identical rows with identical values.
- row-key references.
- compound references.
- escaped colons in compound reference values.
- multiple row errors with `failFast=false`.
- first row error with `failFast=true`.

The demo tests cover:

- import controller routing and HTTP response status.
- absence of public `references` in the response.
- full integration import through `CoredeuxImportService` and the JPA-backed demo application.

Run the relevant reactor slice:

```
mvn -pl modules/coredeux-import,modules/coredeux-spring-boot-demo -am test
```

Common Failure Cases

Missing Unique Column

Request:

```
{
  "operation": "MODIFY",
  "entity": "com.coredeux.demo.domain.Product",
  "columns": [
    { "name": "sku" },
    { "name": "name" }
  ],
  "rows": [
    {
      "values": {
        "sku": "SKU-1",
        "name": "Updated"
      }
    }
  ]
}
```

Response log:

```
At least one existing-entity resolution strategy is required for operation:
MODIFY.
Configure unique columns, lookup, or query.
```

Missing Field

If a column maps to a field that does not exist on the POJO target, validation fails before row execution.

Bad Enum Value

Enum conversion uses `Enum.valueOf`, so values must match enum constants exactly.

Missing Row-Key Reference

When `reference: "*"` points to a key that has not been stored in the current execution, the row fails.

Unsupported Nested Map Conversion

Map conversion is opt-in through `jsonMapImporterHandler`. That handler supports flat maps with scalar values and rejects nested objects or arrays with a column-level error. The default handler does not convert maps.

Extension Points

Add a Custom Value Handler

Use this when one column needs special parsing, lookup, enrichment, or complex object construction.

Steps:

1. Implement `CoredeuxImportValueHandler`.
2. Register it as a Spring bean with a stable name.
3. Use that bean name in `ImportColumn.handler`.

Add File Parsing

File parsing already lives inside `coredeux-import` today. Extend the module when you need another file format, and let the host application wrap it with upload endpoints, command handlers, or other delivery adapters.

The parser code inside `coredeux-import` can own concerns such as:

- PSV/CSV/XLSX parsing.
- escaping.
- header syntax.

- raw text line parsing.
- richer collection value representation.

The host application can own concerns such as:

- multipart upload handling.
- request authentication.
- route design.
- progress tracking.

Design Notes

- `ImportResponse` exposes logs only; row-key references are internal.
- `statementIndex` and `rowIndex` are one-based in logs.
- `rowIndex` is null for statement-level validation failures.
- `lookup` exists for statement-level existing-entity searches when equality against unique columns is too limited.
- `lookup.column` is optional because the common case should read naturally: search field `sku` from column `sku`. It exists for source/target mismatches, such as searching `emailHash` from an imported `email` value.
- `query` exists as the escape hatch for store-specific searches that cannot be represented by unique columns or structured lookup.
- `{{paramName}}` is only a documentation placeholder style. Runtime query syntax belongs to the configured `CoredeuxDataAccessService`; Coredeux import passes query text plus derived params to `CoredeuxService.query`.
- `mode` controls entity collection writes. `replace` overwrites the collection; `append` adds incoming values to the existing collection; `clear` empties or initializes the collection.
- The current entity target service relies on no-argument constructors.
- The current implementation writes fields directly through Coredeux reflection helpers.
- The service uses `CoredeuxService`, so normal Coredeux hooks, validators, workflows, and audit behavior can apply through the core service path.

Current Caveats And Improvement Ideas

- Default conversion does not support nested object, array, or arbitrary complex type assignment.
- Partial collection removal by value is not implemented.
- Public API examples should stay aligned with the Postman collection.

[Previous: Text And Excel File Import](#) | [Documentation Home](#) | [Next: Coredeux Export](#)

